

Student Information: 阮冠威 - 615461004

彭安瑞 - 614461008

Heterogeneous OS Benchmarking for Real-Time Edge Control on Asymmetric Hardware

Abstract

As industrial maintenance evolves from Predictive Maintenance (PdM), which alerts a human operator, to Prescriptive Maintenance (PsM), which requires the controller to act autonomously within milliseconds, the operating system becomes a critical bottleneck. Linux provides the libraries needed for analytics but cannot guarantee consistent timing under load; a Real-Time Operating System (RTOS) like Zephyr guarantees timing but lacks compute resources for heavy analytics. This project benchmarks a Heterogeneous Dual-OS Architecture on the Arduino UNO Q, splitting analytics onto Linux and real-time actuation onto Zephyr RTOS, against two monolithic Linux baselines across 10,000 control-loop cycles under increasing stress, measuring deadline miss ratios, worst-case latency, and jitter.

Introduction

1.1. Motivation

In industrial automation, machinery relies on fast control loops. For example, a motor controller may need to send speed adjustments every 10 ms. Traditional **Predictive Maintenance (PdM)** analyzes sensor data and warns a human operator; a few seconds of delay is acceptable. **Prescriptive Maintenance (PsM)** removes the human entirely: the system detects a problem, computes a corrective action (e.g., reduce motor speed by 15%), and executes it autonomously within milliseconds. This closed-loop approach demands hard real-time guarantees that PdM never needed.

1.2. The Operating System Problem

Running PsM on a single controller creates conflict:

- **Linux alone:** Rich libraries for FFT and AI inference, but its CFS scheduler and virtual memory cause unpredictable delays under load. A 30 ms scheduling stall during a 10 ms control cycle means the motor continues at an unsafe speed.
- **RTOS alone:** Zephyr guarantees timing, but the MCU's 786 KB SRAM cannot host heavy analytics.

- **Patched Linux (SCHED_FIFO):** Reduces average latency but cannot fully eliminate page faults and kernel contention that cause rare worst-case spikes.

1.3. Our Approach

We use the **Arduino UNO Q**, which has two processors on one board. Linux (Cortex-A53) handles analytics; Zephyr RTOS (Cortex-M33) handles the 10 ms motor-control loop. They exchange data through shared-memory IPC (inter-processor communication). Because the processors are independent, Linux delays cannot interfere with Zephyr's real-time path.

We compare three configurations:

Config	Scheduler	Analytics	Actuation
A	Linux CFS	Linux	Linux
B	Linux SCHED_FIFO	Linux	Linux
C	Linux + Zephyr	Linux	Zephyr RTOS

Each runs 10,000 cycles under idle, moderate, and worst-case stress (stress-ng). We measure deadline miss ratio, worst-case latency, jitter, and kernel telemetry (context switches, page faults).

1.4. Goals

- Show quantitatively how Linux's OS mechanisms cause deadline misses and how the dual-OS architecture avoids them.
- Use our experimental results to illustrate OS concepts from the course (e.g., how CFS scheduling, page faults, and priority inversion directly cause late control commands).
- Answer the practical question: when is a single Linux system good enough for real-time control, and when do you need a separate RTOS?

1.5. Report Organization

The rest of this report is organized as follows:

- Section 2 - Background & Syllabus Mapping (OS theory review).
- Section 3 - System Architecture (hardware, dual-OS setup, IPC).
- Section 4 - Experimental Setup (configurations, stress levels, metrics).
- Section 5 - Results & Analysis (latency data, deadline misses).
- Section 6 - Design Guidelines (Summarize and practical recommendations).
- Section 7 - Conclusion & Future Work